Enmatica
by
Villainous Softworks
(Placeholder - replace with LOGO)

Scalars

These are the base types in **Enmatica** used to store a single data value. They represent how the data is stored in memory and follows the standard used in C++. They are classified as the following,

- *Boolean*
- *Integers*
- *Unsigned Integers*
- *Floating Point Numbers*

Boolean:

A *boolean* is a data type used to represent two values, **true** or **false**. It is used for decision making and logical operations. They are represented using numbers in computers. The value is ***false*** if the bits of the data are all zeros, that is, **0000 0000** (8-bit). Otherwise the value is considered to be ***true***, preferably the value one, **0000 0001** (8-bit). It can be represented by using a single bit, however each data in a computer has to be addressable. Hence a boolean is stored using the lowest possible addressable data, that is, 8-bit. It can also be stored using larger sizes but it would be a waste of space, unless absolutely required like for alignment. In Enmatica, the boolean data types available are `bln8`, `bln16`, `bln32` and `bln64`, where the number following `bln` shows the number of bits each type occupies. `bln32` is used in the Vulkan graphics API.

Integers:

An integer is a data type used to represent numbers, positive as well as negative. The most significant bit **(MSB)** is the sign bit used to identify whether the given number is positive or negative. The number is positive if the MSB is 0, otherwise the number is negative, that is the MSB is 1. The negative numbers are stored in the 2's complement form. For example, the number **1** is stored as **0000 0001** (8-bit) and the number **-1** is stored as **1111 1111** (8bit). In **Enmatica**, the integer data types available are `int8`, `int16`, `int32` and `int64`. The possible minimum and maximum values represented by each types are given below,

`int8`	(8-bit)	-	-128 to 127
`int16`	(16-bit)	-	-32768 to 32767
`int32`	(32-bit)	-	-2147483648 to 2147483647
`int64`	(64-bit)	-	-9223372036854775808 to 9223372036854775807

Unsigned Integers:

An unsigned integer is a data type used to represent whole numbers, that is positive integers including zero. Since there's no need for an extra bit for representing the sign, these can represent numbers twice the range for the same amount of size occupied on disk. In **Enmatica**, the integer data types available are `uin8`, `uin16`, `uin32` and `uin64`. The possible minimum and maximum values represented by each types are given below,

`uin8`	(8-bit) -	0 to 255
`uin16`	(16-bit) -	0 to 65535
`uin32`	(32-bit) -	0 to 4294967295
`uin64`	(64-bit) -	0 to 18446744073709551615

Floating Point Numbers:

A floating point number is a data type used to represent numbers, be it fractional numbers or integers. In most computers, it follows the IEEE-754 standard and the commonly used among those are binary16, binary32, binary64 which are commonly referred to as half, float and double respectively. In **Enmatica**, the floating-point numbers are available as `flt16`, `flt32` and `flt64`. The number represented by the floating-point value can be given by the following formula,

$$(-1)^{s}\left(1+\sum_{i=1}^{f} b_{f-i}2^{-i}\right)\left(2^{e_v-\left(2^{e-1}-1\right)}\right)$$

where,

s – value representing the sign part of the number. It shows whether the number positive(0) or negative(1). It takes exactly 1-bit for all sizes.

f – value representing the fractional part of the number. It occupies 10-bit, 23-bit and 52-bit for half, float and double respectively.

e_v – value representing the exponential part of the number. It occupies 5-bit, 8-bit and 11-bit for half, float and double respectively.

$2^{e-1}-1$ – exponent bias that is subtracted from the given exponential value.

There are also special values like infinity and Not a Number(NaN). If the exponent bits are zero and if the fraction is zero then the number is zero otherwise the number is subnormal. If the exponent bits are one and if the fraction is zero then the number is infinity otherwise the number is NaN.

Vectors

- *Vectors – 32-bit to 128-bit*

  vec2(vec64), vec3(vec96) and vec4(vec128)

  bool, int32, uin32, flt32 and flt64

  Vectors can be of any of the above-mentioned scalar types.

  Only vec96(vec3) occupies more than the specified size(Uses 128-bit; Specified 96-bit) to match alignment requirements.

- *Matrices – 32-bit to 512-bit*

  mat2×2, mat2×3 and mat2×4

  mat3×2, mat3×3 and mat3×4

  mat4×2, mat4×3 and mat4×4

- *Quaternions*

  quat